Codefest 9

Hardware for Artificial Intelligence and Machine Learning (HW4AI)

ECE 410/510

Spring 2026

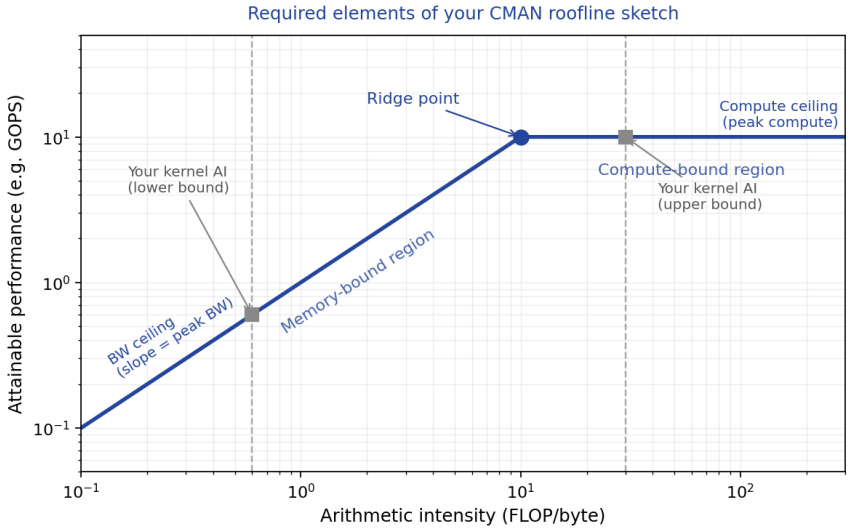
CMAN — Arithmetic intensity of your project kernel from first principles

No Monday lecture this week (Memorial Day). **Both challenges draw on material from Weeks 2, 3, and 8. No new concepts are introduced.**

Do not use a profiler for this challenge. Compute the arithmetic intensity of your project's dominant kernel analytically.

Tasks

1. Identify the dominant kernel in your accelerator (e.g., matrix multiply, convolution, dot product). State its dimensions and data types precisely.
2. Count total FLOPs (or MACs×2) for one invocation of the kernel at your design's operating point.
3. Count total bytes transferred from off-chip memory assuming no data reuse (lower bound on AI). Count total bytes transferred assuming perfect on-chip data reuse for weights (upper bound on AI). Both calculations must show the formula and values. If your kernel does not fit the GEMM-style weight-reuse pattern (e.g., depthwise convolution, embedding lookup, streaming sparse MVM, event-driven sparse compute), state the reuse pattern that actually applies to your kernel and compute the two bounds against that pattern instead. Name the pattern explicitly.
4. Compute arithmetic intensity for both bounds. Draw a hand-drawn roofline sketch using the spec of your target synthesis platform (sky130 PDK nominal figures for an ASIC target, or your FPGA datasheet figures if targeting an FPGA). If you are still benchmarking against your M1 software baseline, also sketch the baseline platform's roofline on the same plot for comparison. Mark both AI bounds and your kernel's attainable performance range. Save as `codefest/cf09/cman_roofline_sketch.pdf` (or .png).
5. State:
    - Is your design currently limited by your hardware interface bandwidth, on-chip memory bandwidth, or compute units?
    - What is the single highest-leverage change to improve performance at this point?
    -



Your sketch must include: labeled axes, BW ceiling, compute ceiling, ridge point, and your kernel's two AI bounds. Use the actual numbers from your target platform and your kernel.

Figure 1. Roofline structure required for the CMAN sketch.

CMAN NO AI	Deliverable: `codefest/cf09/cman_ai_analysis.md` (or .pdf/.png) containing • kernel dimensions and data type; • FLOPs count with formula; • two AI calculations (no-reuse and full-reuse bounds) with formulas; • hand-drawn roofline sketch saved as `codefest/cf09/cman_roofline_sketch.pdf`; • bottleneck identification and improvement suggestion.

CLLM — Accelerator benchmarking vs. software baseline

Run your M1 software baseline again and compare it against your current hardware accelerator implementation. Use simulation, FPGA, or cycle-accurate simulation. If simulation is too slow for a full benchmark, use a representative workload and extrapolate — document the extrapolation clearly.

Tasks

6. Run your M1 software baseline on the same hardware as M1. Record: execution time (ms), throughput (samples/sec or FLOPs/sec), memory usage.
7. Run your current accelerator (in simulation via cocotb, on FPGA if available, or cycle-accurate simulation). Record the same metrics. If no end-to-end simulation is currently runnable (e.g., your M3 work resulted in a justified scope adjustment, or your testbench is not yet passing), use the fallback path: report a projected peak throughput computed from your synthesis results as (clock frequency × useful operations per cycle), and project memory bandwidth from your interface specification. Document the projection assumptions explicitly. Projected numbers must be labeled as such everywhere they appear in this codefest and in M4.
8. Compute speedup (throughput ratio) and energy efficiency improvement (if energy data is available from synthesis). Record results in a table in `codefest/cf09/benchmarks/benchmark_results.md`.
9. Plot your accelerator on the roofline using measured or projected throughput and your C1 arithmetic intensity. Label the point as “measured” or “projected” according to which path you used in task 2. Save as `codefest/cf09/benchmarks/roofline_plot.png`. In codefest/cf09/benchmarks/roofline_analysis.md (100–150 words), diagnose any gap between where the accelerator landed and where you expected it. If you used the projected path, the analysis must instead identify the dominant uncertainty in the projection and what would be needed to convert it to a measurement.
10. List the three most important remaining changes before M4 in `project/remaining_tasks.md`. Be specific: not “improve performance” but “replace the 32-bit adder on the critical path with a carry-save adder to reduce the 2.4 ns delay.”

CLLM AI OK	Deliverable: GitHub commit containing • `codefest/cf09/benchmarks/benchmark_results.md` (table with SW baseline and HW accelerator rows, speedup computed); • `codefest/cf09/benchmarks/roofline_plot.png` with accelerator point marked; • `codefest/cf09/benchmarks/roofline_analysis.md` ≥100 words explaining any gap; • `project/remaining_tasks.md` with ≥3 specific (non-generic) tasks.